

AI for Economic Research: Dynamic Models, Language, and Agents

Lecture 8.4b: Demo, Failure Modes, Evaluation, Bridge to Agentic

Zhigang Feng

Spring 2026

Topic 8.4b Agenda

1. **Classroom Demo: OLG RAG** — a working pipeline you can run today
2. **Failure Modes and Mitigations**
3. **Evaluation** — RAGAS and Cohen's κ
4. **From RAG to Agentic RAG** — bridge to Lecture 9
5. **Summary and Closing**

*Arc: T1 Prompting wall → T2 Chunking & embeddings → T3 Vector & graph retrieval → T4a Augmented generation → **T4b Demo, failures, agentic bridge***

Classroom Demo: OLG RAG

Demo Step 1 — The Query and the Pipeline

Classroom question: *What did Diamond (1965) add to the OLG approach?*



- **Why this question.** It is a classic methodology question (what does the Diamond extension contribute to Samuelson-style OLG?) — answerable from the source paper if and only if retrieval finds the right paragraph.
- **Source (local copy):** `./demos/M4_rag_olg_demo`
- **Default path is offline:** no API key, no network, no vector DB service.
- **Teaching point:** citations are an answer contract, not decoration. Each demo step makes one part of the contract visible.

Demo Launch and Expected Artifacts

```
cd demos/M4_rag_olg_demo
/usr/bin/python3 run_demo.py
/usr/bin/python3 run.py ask "What did Diamond 1965 add to the OLG approach?"
PYTHONPATH=src /usr/bin/python3 -m unittest discover -s tests
```

Artifact	What students should inspect
chunk file	boundaries and line references
retrieval report	top- K evidence, scores, and anchor checks
extractive answer	whether every claim is supported by retrieved text
unsupported query	whether the system refuses rather than improvises

Files are written under the demo's `build/` folder, including `chunks.json` and a retrieval-report CSV.

Demo Step 2 — Corpus Audit Before You Index

- Before chunking, *look at the corpus*. The OLG demo's `run_demo.py` prints a Step-1 audit:
 - character count, line count, heading count
 - the first ~ 5 headings (sections of Spear-Young's paper)
- **Why this matters.** If the audit shows zero headings, your chunker will be flying blind: no section boundaries, no metadata to filter on later. The corpus audit is the cheapest sanity check in the entire RAG pipeline.
- **Practical rule.** If you cannot describe your corpus in two minutes from an audit print-out, you are not ready to index it.
- **Output to inspect:** `build/chunks.json` after Step 2 of `run_demo.py`. Each chunk carries `chunk_id`, `citation`, and `section` — the metadata that makes retrieval auditable.

Demo Step 3 — Retrieval Diagnostics + Anchor Checks

What the demo prints (per question)

- top- K chunks with score
- citation (file + line range)
- section heading
- *anchor hits* — did retrieval surface the gold-standard phrases the instructor expected?
- PASS/FAIL verdict per question

Why anchor checks matter

- Score thresholds alone can lie — a low-quality chunk can still rank highest.
- Anchors are *specific phrases* that must appear in a correct retrieval.
- Anchor hit rate is the simplest cousin of context-recall (T4b RAGAS).
- Watching the PASS/FAIL trail tells you which question types your index handles — and which it doesn't.

Rule restated: read the retrieval report before the generated answer. The diagnostic is the publication-grade artifact.

Demo Step 4 — The Grounded Prompt Anatomy

Prompt part	What it does in the OLG demo
Role	"You are a research assistant for academic economists."
Context	retrieved chunks, each prefixed with <code>[filename:La-b]</code>
Question	the user query, verbatim — not rephrased
Citation rule	every substantive claim must carry one <code>[filename:La-b]</code>
Refusal rule	"If the context does not support the answer, say so explicitly."

- **Why preserve line ranges.** An answer that cites `[Spear-Young_OLG:L412-440]` is one-click-verifiable; a free-form "the paper argues. . ." is not.
- **Lec. 7 callback.** The *grounded generation contract* from Lec. 7 T3b ("cite-then-claim") is exactly what this prompt enforces, line by line.
- **Try it (next slide).** Run `run_demo.py`; inspect Step 6 of the script for the prompt that the model actually sees.

Demo Step 5 — Extractive Answer vs. Refusal

Supported query (Q3 in demo)

- Top retrieved chunks discuss Diamond's production-side extension of Samuelson OLG.
- Extractive answer quotes those chunks verbatim, with line citations.
- Anchor hits ≥ 1 ; PASS.
- This is what a publication-grade RAG should look like.

Unsupported query (refusal test)

- E.g. "What is the demo author's email address?"
- Top retrieved chunks have low score and no anchor coverage.
- Extractive answer returns: *"I do not have enough retrieved evidence in the provided source to answer."*
- This is the *feature*, not the bug.

The two behaviors are inseparable. A system that can answer "Diamond 1965" correctly *and* refuses to invent an email address is the minimum bar for using RAG in published research.

Artifact: `build/terminal_retrieval_report.csv` — archive it next to your paper draft.

Failure Modes and Evaluation

When RAG Fails

Six common failure modes:

1. **Retriever miss.** Relevant document exists but the embedder ranks it outside top- K . Often caused by terminology mismatch.
2. **Re-ranker promotes the wrong context.** Cross-encoder mis-scores; the LLM gets a confident-but-irrelevant chunk.
3. **Generator ignores the retrieved context.** The LLM falls back on its parametric memory and answers from training data — often confidently and incorrectly.
4. **Chunk boundary cuts critical information.** The fact spans the boundary between two chunks; neither chunk alone contains the answer.
5. **Outdated index.** The corpus has been updated, but the index hasn't been re-run. The retriever returns stale chunks.
6. **Ambiguous query.** “Inflation” could mean CPI, PCE, expectations, or asset prices. The retriever picks one interpretation; the user wanted another.

Mitigations — One per Failure Mode

- **Retriever miss** $\Rightarrow$ *query expansion* and *HyDE* (hypothetical document embeddings: ask the LLM to draft a fake answer, embed *that*, retrieve nearest documents). Also: hybrid sparse + dense retrieval.
- **Re-ranker mis-scores** $\Rightarrow$ try a stronger re-ranker (`bge-reranker-v2`); train a domain-specific re-ranker on labeled query-document pairs.
- **Generator ignores context** $\Rightarrow$ stricter system prompt (“Answer ONLY from the context”); lower temperature; faithfulness check in post-processing.
- **Chunk boundary cuts information** $\Rightarrow$ semantic chunking; larger chunk overlap; or use a parent-document retriever (retrieve a chunk, return its parent paragraph).
- **Outdated index** $\Rightarrow$ incremental ingestion pipeline triggered when source documents change; nightly re-embed for high-churn corpora.
- **Ambiguous query** $\Rightarrow$ query rewriting (ask the LLM to clarify or expand the query before retrieval); multi-query retrieval (generate N paraphrases, retrieve for each, fuse).

Evaluation: The RAGAS Framework

- Es, James, Espinosa-Anke & Schockaert (2023), *EACL*, “RAGAS: Automated Evaluation of Retrieval Augmented Generation.” Now the standard.
- **Four metrics, two for retrieval and two for generation:**
 - **Context precision.** Of the retrieved chunks, how many are actually relevant to the query? (*Retriever quality.*)
 - **Context recall.** Of the chunks that should have been retrieved, how many did we get? (*Retriever coverage.*)
 - **Faithfulness.** Are all the claims in the answer supported by the retrieved context? (*Generator anti-hallucination.*)
 - **Answer relevance.** Does the answer actually address the query? (*Generator on-topic.*)
- RAGAS computes all four automatically using an LLM-as-judge. Cost a few cents per query.
- **The right way to use it:** build a small (~50-question) eval set for your specific corpus, run RAGAS each time you change a pipeline component, and only ship a change when the metrics improve.

Cohen's κ for RAG: Beyond LLM-as-Judge

- **RAGAS uses an LLM as the judge** — fast and useful for iteration, but inherits exactly the drift, sycophancy, and version-instability that Lec. 7 T3b documented (Yin–Vu–Persico replication, $\kappa = 0.36$ across frontier LLMs on the same rubric).
- **For publication-grade research**, layer a human-anchored audit on top:
 1. Build a 100–200-item **gold standard**: queries paired with ideal retrievals *and* ideal answers, curated by a domain expert.
 2. Have a *second annotator* score retrieval relevance independently; compute **Cohen's κ** (target ≥ 0.75 , per Lec. 7's measurement-rigor frames).
 3. Treat RAGAS as the fast inner loop, the gold-standard κ audit as the slow outer loop. Only publish numbers from the latter.
- **AEA reproducibility (Oct 2024) extends to RAG pipelines.** Disclose: prompt template, embedding model + version, vector DB + parameters, retrieval K , generator model + version + temperature, evaluator model + version, gold-set provenance.
- **Take-away:** RAGAS tells you whether the pipeline got *better*; Cohen's κ tells you whether the pipeline is *good enough to cite*.

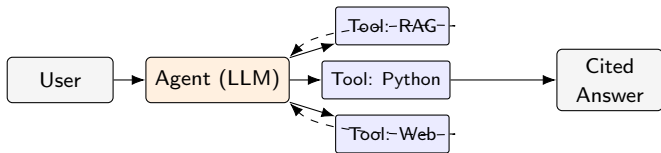
Open Challenges

- **Multi-hop reasoning.** Questions that require synthesizing evidence across *multiple* retrieved documents (“Did the Fed’s response in 2008 differ from the ECB’s in 2011?”). Current RAG often retrieves either one side or the other, not both. Active research area.
- **Long-document QA.** Documents that are themselves longer than the context window (a 600-page legal opinion). Hierarchical retrieval and recursive summarization help but aren’t solved.
- **Tables and figures.** Most embedders are trained on prose. Tables and figures in PDFs are routinely missed or garbled. Specialized layout-aware extractors (nougat, marker) help.
- **Multilingual retrieval.** Retrieving from a corpus that mixes English, French, Spanish, Mandarin... requires multilingual embedders (multilingual-e5-large, bge-m3). Quality is uneven across languages.
- **Dynamic knowledge and graph maintenance.** A flat vector index is easy to append to. A graph-based index may require re-extracting entities, re-computing communities, and re-writing summaries each time new documents arrive.
- **Evaluation cost.** RAGAS-style LLM-as-judge eval is expensive at scale. Cheap, reliable retrieval metrics for domain-specific corpora remain an open problem.

Bridge to Lecture 9

From RAG to Agentic RAG: Retrieval as One Tool Among Many

- **Single-shot RAG** (today's pipeline): fixed plumbing — query → retrieve → augment → generate. One round; the developer wires the steps.
- **Agentic RAG**: the LLM *decides* whether, when, and what to retrieve, via the *JSON tool calls* introduced in Lec. 7 T3a (“Structured Output: JSON as a Measurement Tool”). The loop continues until the model signals done.
- **Patterns this unlocks**: multi-hop retrieval (one query begets the next); *query rewriting* (the agent reformulates an ambiguous user question before retrieving); and *cross-tool composition* — RAG *plus* code execution *plus* web fetch *plus* structured-data queries.
- **Inherited failure modes (Lec. 7 T5)**: the agent's context still drifts; sycophancy still bites. Worse: an agent can retrieve confirming evidence *and ignore disconfirming evidence* unless the loop is designed to penalize that.



Summary: The Four-Step Pipeline + the Why

1. **Chunk.** Split documents into self-contained, metadata-rich pieces.
2. **Embed.** Map each chunk to a vector so semantically similar text lives nearby in $\mathbb{R}^d$.
3. **Store & Retrieve.** Use a vector DB (Chroma, Qdrant, FAISS) for local semantic search; add graph or hierarchical indexes when the task depends on relationships, coverage, or multi-hop reasoning.
4. **Augmented Generation.** Insert the retrieved evidence into a prompt template that demands grounded, cited, refusable answers.

Why this matters: A vanilla LLM hits a hard wall on freshness, scale, latency, cost, and attribution — exactly the dimensions that matter most for academic research. RAG separates the cost of indexing from the cost of querying, giving you a system that is cheap to update, cheap to query, faithful to sources, and capable of refusing to answer.

The LLM is the brain. The index is the memory. The retriever is the librarian.

Closing — Pick the Right Tool, Disclose Like a Researcher

RAG turns institutional text — FOMC, NBER, Congress, SEC, central banks — into a **queryable research instrument**.

Decision	Pick this when. . .
RAG (today)	you need facts from a corpus the LLM never saw, with citations the reader can verify.
Fine-tuning	you need <i>style</i> , <i>format</i> , or rare <i>tokens</i> — not facts. (Pair with RAG; don't substitute.)
Agentic RAG (Lec. 9)	the question requires multi-hop retrieval, query rewriting, or cross-tool composition (RAG + code + web).
None of the above	the corpus fits in 200K tokens <i>and</i> the question is single-shot — a long-context prompt suffices.

Disclosure checklist for publication (AEA 2024 + Cohen's κ frame):

- embedding model + version, vector DB + parameters, retrieval K , re-ranker (if any) + version
- prompt template, generator model + version + temperature, evaluator model + version